



Opisi algoritama

Zadatke, testne primjere i rješenja pripremili: Dominik Fistrić, Krešimir Nežmah, Ivan Paljak i Paula Vidas. Primjeri implementiranih rješenja su dani u priloženim izvornim kodovima.

Zadatak Mađioničar

Pripremila: Paula Vidas

Potrebno znanje: palindromi

Prvi podzadatak možemo riješiti binarnim pretraživanjem. Primijetimo da, ako je neka riječ palindrom, onda je i riječ dobivena micanjem prvog i zadnjeg slova također palindrom. To znači da, ako postoji palindromski podstring duljine $l > 1$, onda postoji i palindromski podstring duljine $l - 2$ (ali ne nužno i duljine $l - 1$). Stoga, možemo binarnim pretraživanjem odvojeno naći najdulji parni i najdulji neparni palindromski podstring, te je odgovor veći od ta dva. Za to nam treba manje od $2 \cdot \lceil \log(n/2) \rceil \cdot n$ upita.

Drugi podzadatak riješit ćemo koristeći najviše $3n$ upita. Za svaki r ($1 \leq r \leq n$), naći ćemo najmanji l za koji je podstring $[l, r]$ palindrom (drugim riječima, $[l, r]$ je najdulji palindromski podstring koji završava r -tim slovom). Na početku stavimo $l = 1$, te idemo redom $r = 1, r = 2, \dots, r = n$. Kada prelazimo s $r - 1$ na r , primijetimo da novi l može biti najmanje $l - 1$ (jer micanjem prvog i zadnjeg slova palindrom ostaje palindrom). Dakle, pri prijelazu prvo provjerimo je li podstring $[l - 1, r]$ palindrom. Ako je, smanjujemo l za 1 te smo našli željeni l . Inače, sve dok podstring $[l, r]$ nije palindrom, povećavamo l za 1. Primijetimo da je uvijek $1 \leq l \leq n$ te se tijekom algoritma l može smanjiti za 1 najviše n puta, pa se stoga može povećati za 1 najviše $2n$ puta, što daje najviše $3n$ upita.

U trećem podzadatku, kada se riječ sastoji od najviše dva različita slova, možemo s $n - 1$ upita otkriti riječ, tako što postavimo upit za svaki podstring duljine dva. Ako je palindrom, onda su ta dva slova jednaka, inače su različita. Nakon što znamo riječ, najdulji palindromski podstring možemo naći primjerice pomoću binarnog pretraživanja i *hashiranja* ili koristeći Manacherov algoritam.

Četvrti podzadatak rješavamo koristeći najviše $2n$ upita. Odvojeno ćemo naći najdulji neparni i najdulji parni palindromski podstring. Riješimo prvo neparni slučaj. Neka je na početku $d = 1$, to će nam predstavljati duljinu najduljeg do sad nađenog neparnog palindromskog podstringa. Idemo redom $i = 1, i = 2, \dots, i = n$. Za svaki i radimo sljedeće: sve dok je podstring duljine $d + 2$ sa "središtem" u i (to je podstring $[i - (d + 1)/2, i + (d + 1)/2]$) palindrom, povećavamo d za 2. Naravno, ako bi taj podstring izlazio izvan granica riječi, prestajemo i prelazimo na sljedeći i . Analizirajmo sada broj upita. Postavili smo $(d - 1)/2$ potvrdnih upita (kada je podstring bio palindrom) te najviše $n - (d + 1)/2$ negativnih upita (za svaki $i \leq n - (d + 1)/2$ najviše jedan, a za veće i nismo pitali nijedan upit jer je podstring izlazio izvan granica). To ukupno daje najviše $n - 1$ upit. Parni slučaj rješavamo analogno.



Zadatak Mensza

Pripremio: Ivan Paljak

Potrebno znanje: prikaz prirodnog broja u binarnom zapisu

Fokusirajmo se najprije na prva dva podzadatka kojima smo mogli pristupiti na “direktan” način. Odnosno, moguće je osmisliti postupak kojim će Alojzije i Benjamin u nizove a i b enkodirati tko su i koji su broj vidjeli u uredu Gospodina Malnara.

U prvom podzadatku je postupak enkodiranja relativno jednostavan. Primjerice, Alojzije je mogao svoj identitet i broj A enkodirati nizom koji se sastoji od $2A$ jedinica, a Benjamin je mogao svoj identitet i broj B enkodirati nizom koji se sastoji od $2B - 1$ dvojki. Budući da Alojzije i Benjamin nikad neće imati zajednički broj u svojim nizovima, te da im se nizovi sastoje od uzastopnog ponavljanja jednog broja, zaključujemo da će se niz c sastojati od dva broja, a to su upravo $2A$ i $2B - 1$. Cecilija će tada parni element niza c podijeliti s 2 kako bi dekodirala broj A , te će na sličan način dekodirati i broj B , pa samim time jednostavno odrediti koji je veći.

Na sličan način ćemo pristupiti i drugom podzadatku, samo ćemo osmisliti nešto efikasniji način enkodiranja brojeva A i B . Poslužiti ćemo se pritom zapisom brojeva u binarnom sustavu, te ćemo ideju iz prethodnog odlomka primijeniti na svaki bit zasebno. Preciznije, Alojzije će za i -ti postavljen bit u binarnom zapisu broja A u niz a dodati broj $2i + 2$ točno $2i + 2$ puta. Slično, Benjamin će za j -ti postavljen bit u binarnom zapisu broja B u niz b dodati broj $2j + 1$ točno $2j + 1$ puta. Lako je primijetiti da će Cecilija na temelju parnosti odrediti koji brojevi odgovaraju bitovima broja A , a koji brojevi bitovima broja B .

Ograničenja za posljednji podzadatak su prestroga za dosadašnju strategiju. Odnosno, morat ćemo odustati od ideje će Cecilija rekonstruirati brojeve A i B .

Promatrat ćemo i dalje brojeve u binarnom zapisu i zapitati se što vrijedi kada je $A > B$. Vrijedi da brojevi A i B imaju neki zajednički prefiks binarnih znamenaka, a sljedeća znamenka je jedinica u broju A i nula u broju B . Primijetite da je pozicija tog bita jedinstvena, odnosno postoji samo jedna pozicija takva da se na toj poziciji u broju A nalazi jedinica, u broju B nula, a prefiks do te pozicije je jednak u oba broja. Nazovimo taj bit *bitnim*. Promatrajmo sve pozicije gdje bi se mogao nalaziti bitni bit. Naravno, u broju A to su sve pozicije gdje se nalazi jedinica, a u broju B to su sve pozicije gdje se nalaze nule. Ideja je na temelju broja A generirati skup svih prefiksa do pozicije gdje bi se mogao nalaziti bitni bit, te istu stvar napraviti za broj B , samo što ćemo tamo invertirati bitni bit. Primijetite da, ako je $A > B$, u nizu brojeva koje generiramo temeljem broja A i nizu brojeva koje generiramo temeljem broja B , postoji točno jedan zajednički element. Ako je $A < B$, lako se uvjeriti da ne postoji zajednički element.

Dakle, Alojzije će najprije pretvoriti broj A u binarni zapis, te u niz a zapisati sve prefikse koji završavaju jedinicom, a sufiks popuniti nulama. Benjamin će najprije pretvoriti broj B u binarni zapis, te u niz b zapisati sve prefikse koji završavaju nulom, ali će tu nulu pretvoriti u jedinicu i ostatak sufiksa popuniti nulama. Cecilija će zaključiti da je $A > B$ ako a i b imaju zajednički element.



Zadatak Povjerenstvo

Pripremio: Krešimir Nežmah

Potrebno znanje: topološko sortiranje, strongly-connected komponente

Jasno se nameće interpretacija situacije iz zadatka kao usmjereni graf. Svaka od N osoba predstavljena je čvorom, a ako osoba b ide na živce osobi a , to ćemo predstaviti usmjerenim bridom iz a u b . Uvjet iz zadatka nam tada govori da u danom grafu ne postoji usmjereni ciklus neparne duljine, a potrebno je pronaći skup čvorova tako da nikoja dva čvora iz skupa nisu susjedna te da svaki čvor izvan skupa pokazuje u neki unutar skupa. Ispostavlja se da rješenje uvijek postoji. Opisat ćemo kako konstruirati traženi skup za svaki podzadatak.

U prvom podzadatku dani graf nema usmjereni ciklus, dakle riječ je o DAG-u. Promatrajmo čvorove kojima je out-degree jednak 0, tj. one čvorove koji nemaju izlaznih bridova. Nijedan takav čvor ne može biti izvan povjerenstva jer onda ne bi mogao pokazivati u neki čvor unutar povjerenstva. Prema tome, svi takvi čvorovi nužno moraju pripadati povjerenstvu, a nakon što odaberemo te čvorove, znamo da ne smijemo odabrati čvorove koji u njima pokazuju. Čvorove koje u njima pokazuju zato možemo izbrisati iz grafa te na preostalom grafu ponavljati postupak. Spomenutu ideju možemo implementirati u složenosti $O(N + M)$ pomoću *queuea* u kojeg ubacujemo čvorove kada im out-degree postane 0.

U drugom podzadatku dani graf nema neparni ciklus pa je bipartitan. Slično kao i u rješenju za prvi podzadatak, možemo micati čvorove dok god postoji barem jedan čvor čiji je out-degree jednak 0. Nakon toga preostaje graf u kojem svaki čvor pokazuje u barem jedan drugi čvor i tada je dovoljno obojati čvorove u dvije boje (što je po pretpostavci moguće) te izabrati sve čvorove jedne boje kao dio povjerenstva. Zbog bipartitnosti se neće dogoditi da postoji veza unutar povjerenstva, a svaki čvor izvan povjerenstva će pokazivati u čvorove u povjerenstvu te će ih po pretpostavci biti barem jedan.

Graf u kojem je moguće od svakog čvora doći do svakog drugog nazivamo *strongly-connected*. Svaki je graf moguće particionirati na strongly-connected komponente, a ako svaku komponentu kompresiramo u jedan čvor dobivamo DAG. Dekompozicija se može provesti koristeći poznate algoritme za rastav na strongly-connected komponente, poput Kosaraju-ovog algoritma. Ključna opservacija za treći i četvrti podzadatak je da graf nema usmjereni ciklus neparne duljine ako i samo ako je svaka strongly-connected komponenta bipartitna (gledajući bridove kao neusmjerene). U sljedećem je paragrafu dokaz spomenute tvrdnje.

Ako graf ima neparan usmjereni ciklus, taj ciklus je onda u sadržan unutar jedne strongly-connected komponente. Tada ta komponenta sadrži neparni ciklus čak i ako gledamo bridove neusmjereno pa ona nije bipartitna. Za drugi smjer, pretpostavimo da promatramo strongly-connected komponentu u kojoj je svaki usmjereni ciklus parne duljine. Želimo pokazati da je ta strongly-connected komponenta bipartitna. Pretpostavimo suprotno, da u njoj postoji ciklus neparne duljine te pokažimo da onda postoji i usmjereni ciklus neparne duljine. Svaki brid $u \leftarrow v$ koji je u promatranom neparnom ciklusu okrenut u krivu stranu možemo zamijeniti putom iz u prema v (koji sigurno postoji jer promatramo strongly-connected komponentu). Duljina tog puta je neparna jer zajedno s bridom $u \leftarrow v$ taj put mora tvoriti ciklus parne duljine. Stoga, kada zamijenimo brid s putom, duljina promatranog ciklusa je i dalje neparna. Postupnim zamjenama krivo usmjerenih bridova dolazimo do željene kontradikcije.

Treći podzadatak sada možemo riješiti na sljedeći način. Rastavimo graf na strongly-connected komponente te promatramo bilo koju komponentu čiji je out-degree prema ostalim komponentama jednak 0. Pobjamo tu komponentu u dvije boje te sve čvorove jedne boje uzmemo kao dio povjerenstva, a sve čvorove koje u njima pokazuju izbrisemo. Budući da brisanje čvorova može promijeniti rastav na komponente, nakon svakog ovakvog koraka ponovno radimo dekompoziciju. Ukupna složenost ovog pristupa je $O(N(N + M))$.

Četvrti podzadatak potrebno je riješiti bez da svaki put ponovno radimo dekompoziciju. Rastav na strongly-connected komponente napraviti ćemo jednom na početku te iterirati po komponentama u obrnutom poretку topološkog sortiranja. Kada obrađujemo neku komponentu, moguće je da su neki čvorovi iz nje već izbrisani, no i nakon brisanja promatrana komponenta je bipartitna pa možemo naprosto



iskoristiti rješenje drugog podzadatka. Dakle, kada smo na trenutnoj komponenti, brišemo čvorove dok god postoji jedan s out-degreeom jednakim 0, a nakon toga uzimamo bipartitno bojanje. Ukupna složenost je $O(N + M)$.



Zadatak Vinjete

Pripremili: Dominik Fistrić i Ivan Paljak

Potrebno znanje: DFS obilazak stabla, tournament stablo

Za početak ukorijenimo stablo u čvoru 1. Lako vidimo da je skup potrebnih vinjeta za putovanje do nekog čvora nadskup potrebnih vinjeta za putovanje do roditelja tog čvora. Time dolazimo na ideju da se od čvora 1 spuštamo prema ostalim čvorovima DFS obilaskom te potrebne vinjete za određeni čvor računamo samo preko potrebnih vinjeta roditelja te vinjeta potrebnih za autocestu između čvora i njegovog roditelja. Taj pristup vodi do pravog rješenja, no ovisno o načinu pamćenja vinjeta nekog čvora te spajanja istih s potrebnim vinjetama na autocesti dobivamo rješenja za pojedine podzadatke.

U prvom podzadatku vinjete nekog čvora pamtimo kao niz nula i jedinica gdje jedinica predstavlja da moramo kupiti tu vinjetu. Prolaskom po bridu iteriramo od a_i do b_i te u nizu stavljamo jedinicu na te pozicije. Složenost ovog pristupa je $O(N * M)$ što je dovoljno dobro za prvi podzadatak.

U drugom podzadatku umjesto niza ćemo za intervale vinjeta pamtiti samo početak i kraj svakog intervala. Kada prolazimo po bridu samo ubacujemo novi početak i kraj intervala, a da bi izračunali koliko vinjeta je potrebno u trenutnom čvoru koristimo se jednostavnim sweep-line algoritmom na sortiranim počecima i krajevima. Za spremanje početaka i krajeva intervala možemo koristiti sortiranu strukturu poput Multiset-a te je složenost tada $O(N^2 + N * \log(n))$.

U trećem podzadatku možemo pristupiti slično kao u prvom, no sada umjesto niza brojeva koristimo niz bitova (Bitset). Postupak je isti kao u prvom podzadatku, no kako koristimo Bitset memorijska i vremenska složenost će biti $O(\frac{N * M}{64})$ što je dovoljno za taj podzadatak.

U četvrtom i petom podzadatku ćemo za pamćenje vinjeta koristiti tournament stablo. Operacije koje moramo podržavati su: suma svih jedinica u stablu te postavi sve čvorove u intervalu na jedinice. Dodatni implementacijski detalj koji treba promotriti je vraćanje "unazad", odnosno kada DFS završi s podstablom te se vraća nazad u roditelja (kako bi se eventualno pozvao za neko drugo dijete) potrebno je vratiti se u stanje u kojemu je roditelj originalno bio odnosno povratiti stanje u tournamentu prije prolaska po bridu. Na to je potrebno paziti i u prijašnjim podzadacima, no tamo je relativno lagano vratiti stanje na prijašnje u dobroj složenosti pa prije nije spominjano. Kako bi riješili taj problem možemo koristiti perzistentni tournament koji nam dopušta vraćanja u prošla stanja (<https://wiki.xfer.hr/povijesna/>) ili je moguće pametnije pamtiti stvari u običnom tournamentu kako bi bilo lakše vratiti se unazad (ostavljamo čitatelju za vježbu). Sama složenost algoritma će biti $O(N * \log(N))$ što je dovoljno dobro za oba podzadatka, no treba pripaziti na memorijsku složenost. Ukoliko je tournament implementiran klasično kao niz memorijska složenost će biti $O(M)$ što je dovoljno za četvrti podzadatak, no previše za zadnji. Potrebno je implementirati takozvani sparse tournament (umjesto niza tournament je implementiran preko čvorova s pointerima na djecu) koji zauzima $O(N * \log(N))$ memorije što je dovoljno dobro za sve podzadatke.